TELECOM Paris

IP PARIS

Booting a Linux System

M2 SETI-B4 Embedded Linux

Maria Mushtaq
Maria.mushtaq@telecom-paris.fr
2022-2023

Institut Mines-Télécom

Milestones of the start-up

- Bootloader
- Linux Kernel
- Init process
  - In the rest of this course, we will go into the details of these different steps, for example, a system based on a 32-bit ARM Cortex-A 32 bits processor (representative of many embedded systems)

Plan

- Bootloader
- The interface between the bootloader and the kernel
- Decompressing of the kernel
- Booting the kernel
- Loading an initial memory file system
- Init, the first process
- References

Responsibilities

- In general, the first software layer running on a platform
- Typical responsibilities

  - Basic initialization of the platform
  - Start-up of the other software layers (depending on the case: operating system or main application)
  - Update (of itself or of the software layers)

- Common examples

  - Das U-Boot (embedded)
  - GRUB (PC)
  - Windows Boot Manager (PC)

Before the bootloader

- On some systems, code runs before the bootloader, i.e., on a Cyclone V SoC [1]

- Boot ROM (located in ROM on the Cyclone V)

- Detects the location of the following steps (jumpers)

- Performs the minimal initialization of the HPS

- Loads the next step from the flash to OCRAM (On-Chip RAM)

- Preloader

- Continuation of the HPS initialization

- Initialization of the SDRAM

- Loads the bootloader to the SDRAM and executes it

- Can be done by the Secondary Program Loader (SPL) of U-Boot

Other Example

- Another example, on a PC:

- BIOS/UEFI

- Option ROMs

- Bootloader (Windows Boot Manager or GRUB for example)

Das U-Boot: Presentation

- https://www.denx.de/wiki/U-Boot/
- Extremely flexible and configurable bootloader
- Numerous architectures supported: PPC, ARM, MIPS, x86, m68k, NIOS, Microblaze...
- Reading of an executable/image to be loaded from a mass storage media (SD card, SATA,SPI, I2C, USB...), from a serial link or from the network (TFTP, NFS)
- Support of many file systems (ext2/3/4, FAT, ZFS, JFFS2...)
- Support for device trees (possibility to modify them on the fly)

Das U-Boot

- Possible division into two parts
- Secondary Program Loader

  - Executed from a constrained environment (little RAM for example)
  - Prepares the system for the loading and execution of the rest of the bootloader

- U-Boot itself

The U-Boot
Overview of available controls

- Memory manipulation: md (memory display), mm (memory modify)
- Flash memory manipulation: erase, cp (copy), protect (protect), mtdparts (partition table manipulation)
- Execution: source (script execution), bootm (boot an image from memory), go (starting a standalone application)
- Network : bootp (boot an image via the network with network with BOOTP/TFTP), dhcp (booting an image via network with DHCP/TFTP), loadb (retrieve an image via a serial link)

The U-Boot
Overview of available controls

- Environment : printenv (display environment variables), saveenv (saves the variables), setenv (modifies a variables), run (execute the content of a variable), bootd (execute the content of the variable bootcmd)
- Flattened Device Tree : fdt list (displays a level), fdt mknode (creates a node), fdt set (set the properties of a node), fdt rm (delete a node or a property)

The U-Boot
Example: starting the Linux kernel

- Typical example in an embedded system

  - Loading the kernel in memory from an internal flash or from a file on a partition of an SD card

  - Loading in memory of the file system initial memory

  - Loading of the tree of devices

  - Booting the kernel with the right information

  - Possibility to interrupt the boot sequence by a user action (e.g. typing a character on a serial link for example) to have access to other U-Boot features

Plan

- Bootloader
- The interface between the bootloader and the kernel
- Decompressing of the kernel
- Booting the kernel
- Loading an initial memory file system
- Init, the first process
- References

Starting Linux

Source: https://www.kernel.org/doc/Documentation/arm/Booting

- Seen from the kernel, this is what should be done by the bootloader

  - Initialization of all the RAM of the platform

  - Initialization of a serial port (recommended)

  - Detection of the machine type (mandatory except for support for device trees) to pass it to the kernel

  - Used to select the specific code for platform initialization in the kernel

  - List: https: //www.arm.linux.org.uk/developer/machines/

  - Prepare either the ARM Tags (ATAG) structure or the tree of devices to pass it to the kernel

  - Load the initial memory disk image (optional)

  - Call the kernel image

Booting Linux
ARM Tags structure (ATAG)

- Old method to pass information between the bootloader and the kernel
- List of triples (tag, size, content) placed in memory

  - ATAG_CORE : Beginning of the list (mandatory)
  - ATAG_NONE : End of the list (required)
  - ATAG_MEM : Description of the physical memory (required)
  - ATAG_INITRD2: Location of the initial disk image
  - ATAG_CMDLINE : Kernel command line

- It is recommended to place this structure in the first 16 kB of memory
- The pointer to the beginning of the list is placed in the r2 register

Booting Linux
Peripheral trees

- The device tree is a data structure for describing the hardware components of a system
- Initially designed for workstations and servers based on SPARC architecture
- Proposed by the OpenFirmware project Is now the mandatory structure to describe an ARM-based platform in the Linux kernel (instead of the machine identifier and the ATAG structure)

Booting Linux
Peripheral trees

- Two representations: one in human-readable form (Device Tree Source, DTS) and one easily interpreted by a machine (Device Tree Blob, DTB or Flattened Device Tree)
- In addition to the description of the hardware, the tree can also contain the kernel command line
- The syntax of the trees will be discussed in the following of the course
- It is recommended to place the tree just above the first 128 MB of RAM
- The pointer to the beginning of the list is placed in the r2 register

Booting Linux
Calling the kernel

- It is recommended to place the kernel image in the first 128 MB of RAM, but above the first 32 MB (so that it does not have to be moved before decompression)
- r0 contains 0
- r1 contains the type identifier of the platform
- r2 contains a pointer to the ARM Tags
- (ATAG) or device tree (the kernel identifies if the address corresponds to a tree)

Booting Linux
Calling the kernel

- Pause all devices doing DMA
- All interrupts disabled
- HYP mode (if available) or SVC
- MMU disabled
- Data caches disabled
- Direct jump to the first instruction of the image in ARM mode (except on processors that only support Thumb mode)

Decompressing

- The kernel image may have been generated in two forms: compressed (zImage most often, possibly with a header for U-Boot: uImage) or uncompressed
- In the case of a compressed image, the execution starts in arch/arm/boot/compressed/head.S at the start symbol
- The code has been compiled to be able to run whatever the address at which it has been positioned (PIC : Position Independent Code)

Decompressing

- The code will look for the physical address of the beginning of the RAM. By default, it will take the address contained in pc (physical address of the instruction being executed) and do an AND with the constant 0xf8000000 (the kernel assumes that it has been loaded in the first 128 MB of RAM)
- It adds TEXT_OFFSET (traditionally 0x8000, that is 32 kB) to obtain the final physical address of the .text section of the kernel after decompression

Decompressing

- If possible, it then creates a minimalist page table to activate the I and D caches for the area containing the compressed kernel and the future decompressed kernel (no address translation for the moment)

- It then sets up a stack and a memory allocation area just after the kernel in order to have a minimal support to call code written in C

Decompressing

- It then checks if a device tree (DTB) is concatenated with the compressed image (this is allowed)

  - If the ATAG structure is passed via r2, the tree is updated with the information from the ATAG structure

  - If a device tree is passed via r2, it is ignored and r2 is modified to point to the DTB concatenated

  - The size of the compressed image is updated to take into account the DTB

  - The stack and the area for memory allocation are shifted to not overwrite the DTB

Decompressing

- It then checks if, once decompressed, the kernel will overwrite the compressed image

  - If yes, the code will copy the compressed image (and possibly the concatenated DTB) after the calculated end address of the decompressed image

  - Once moved, it jumps to the new address of the restart symbol (setting up the stack and the allocation area)

Decompressing

- It then calculates the displacement between the address and the address to which the decompression code has been linked
    - It then updates the entries using the Global Offset Table (GOT)
- It initializes the BSS (block starting symbol) field to zero
- It jumps to the decompress_kernel symbol which is a C function in arch/arm/boot/compressed/misc.c and which will manage the decompression

After the Decompressing

- Once the image is decompressed, the code disables and empties the caches
- The code jumps to the __enter_kernel symbol which resets the registers r0 (0), r1 (machine ID) and r2 (ATAG/DTB)
- Then, we finally jump to the beginning of the image (beginning of RAM + TEXT_OFFSET), which corresponds to the text symbol in arch/arm/kernel/head.S

Conclusion on the Decompressing Phase

- In addition to the cultural interest (it doesn't hurt to look at a little assembler from time to time), the knowledge of how the decompression has several practical interests
- Most kernels are compressed, so decompression is an essential step
- A bad positioning of the kernel by the bootloader in memory can lead to a recopy before decompression, which is time consuming

Sources

- arch/arm/boot/compressed/head.S
- Linus Walleij, How the ARM32 Linux kernel decompresses, ao.t 2020, https://people.kernel.org/linusw/how-the-arm32-linux-kernel-decompresses

Plan

- Bootloader
- The interface between the bootloader and the kernel
- Decompressing of the kernel
- Booting the kernel
- Loading an initial memory file system
- Init, the first process
- References

Kernel Startup

- The kernel starts its execution at the stext symbol located in arch/arm/kernel/head.S
- The first task performed by the kernel will be to set up the address translation tables and to activate the MMU
- This operation is a bit complex

Kernel Startup

- The virtual address space is divided into two zones

  - 0x0000_0000 - 0xBFFF_FFFF (3 GB): memory of the running process (userspace)

  - 0xC000_0000 - 0xFFFF_FFFF (1 GB): memory of the kernel (code and data structures)

  (kernelspace)

- The symbol PAGE_OFFSET contains the address of the of the kernel space (0xC000_0000)
- It is possible to change this allocation (but the case shown above is by far the most frequent)

Current Address of Execution

- The kernel code is not position-independent
- It has been generated by the linker to be placed at a certain virtual address (in the area 0xC000_0000 - 0xFFFF_FFFF)
- However, at boot time, the kernel image may have been placed almost anywhere in physical memory (and most probably not at the address provided by the linker)
- The code will therefore try to calculate the displacement between the address at which the code is actually executed and the one at which it has been generated to run

Overview of the low-level initialization process

- Setting up a table of initial pages (covering only the code and data of the placed at virtual address 0xC000_0000)
- Activation of the MMU and jump to the virtual address space
- Setting up the execution environment of the task (PID 0): stack, data structures representing the task
- Initialization of the CPU on which the code is executed
- Initialization of the architecture

  - Hard-compiled code
  - Machine identifier + ATAG
  - Tree of peripherals

Overview of the low-level initialization process

- Identification of available memory regions
- Continuation of the implementation of the virtual memory (direct mapping area)
- First analysis of the kernel arguments
- Implementation of the final memory mapping
- Setting up the table of interrupt vectors
- Initialization of the structures necessary for memory allocation
- The rest of the initialization is globally common to all architectures

Sources

- `arch/arm/kernel/head.S`
- Linus Walleij, *How the ARM32 kernel starts*, août 2020, `https://people.kernel.org/linusw/how-the-arm32-kernel-starts`
- Linus Walleij, *Setting Up the ARM32 Architecture, part 1*, octobre 2020, `https://people.kernel.org/linusw/setting-up-the-arm32-architecture-part-1`
- Linus Walleij, *Setting Up the ARM32 Architecture, part 2*, octobre 2020, `https://people.kernel.org/linusw/setting-up-the-arm32-architecture-part-2`

Continuation of the Kernel Startup

- Initialization of hard-compiled devices in the kernel
- Implementation of the various kernel mechanisms (notably time management, scheduler...)
- Once everything is initialized, it remains now to hand over to the user space for the application startup

Plan

- Bootloader
- The interface between the bootloader and the kernel
- Decompressing of the kernel
- Booting the kernel
- Loading an initial memory file system
- Init, the first process
- References

File System

- To be able to start the applications, you must be able to read executables from a file system
- The kernel must therefore locate the root file system (/) and mount it
- To do this, the kernel must have

  - the location of this root file system (root argument passed to the kernel, examples : /dev/mmcblk0p1 (1st partition of the 1st SD card), /dev/mtdblock0 (flash memory), /dev/sda1 (1st partition of the first SCSI/SATA disk...)

  - Device driver(s) needed to access to the file system

  - The driver for the file system itself (ext2/3/4, fat32...)

File System

- If all the necessary drivers have been hard-compiled into the kernel, no problem
- However, in the case of a distribution that has to handle many different configurations, compile all possible drivers in hard copy, would require a very large kernel
- And so, most often, in this case, the drivers are usually compiled as modules (in order to be able to load only those useful in the given configuration)

Drivers to access the file system root

- If all the necessary drivers have been hard-compiled into the kernel, no problem
- However, in the case of a distribution that has to handle many different configurations, compile all possible drivers in hard copy, would require a very large kernel
- And so, most often, in this case, the drivers are usually compiled as modules (in order to be able to load only those useful in the given configuration)
- But modules are files and to be able to load them, the file system on which they are mounted

Usefulness of an initial file system

- So, in many classical distributions (this is less true in the embedded world since the kernel is often compiled ad-hoc for the target system), the device drivers necessary to access the root file system are not hard-compiled
- So the kernel cannot mount this root file system...
- The solution is an initial root file system (initrd/initramfs)
- The image is loaded in memory by the bootloader and its address is passed to the kernel

Contents of the initial file system

- The kernel mounts this image as the root file system (/)
- This image contains

  - The executable that will be used as the initial process (/init in the case of initramfs) : mandatory (otherwise the kernel will look for the root filesystem elsewhere)
  - Its dependencies (possible shared libraries, dynamic linker and loader : /lib/ld-linux.so, configuration files...)
  - A classic root tree skeleton (/dev, /tmp, /var...)
  - A mechanism for managing special files in /dev
  - The necessary modules for loading the root file system

Contents of the initial file system

- Eventually, this image contains all the files and executables needed to run the system, in which case another root file system is not necessary

Mechanisms

- Two mechanisms: initrd (old) and initramfs (introduced in series 2.6)
- initrd: image (possibly compressed) of a file system (ex. ext2/3, cramfs...)
- initramfs: cpio archive (possibly compressed) which is decompressed by the kernel in a rootfs (memory file system)

initrd
Operation (continued)

- The bootloader places the kernel image and the initrd image in memory
- The kernel creates a memory disk /dev/ram0 and decompresses the initrd image, then releases the memory occupied by it
- If the root device (root parameter) is not /dev/ram0, use the old method called change_root (see below)
- The file system /dev/ram0 is mounted in the root /
- The file /sbin/init is executed with the login user 0 (super-user)

initrd
Operation (continued)

- Traditionally, this executable performs the following operations:

  - Mount the final root file system

  - Exchange root systems so that the final root file system is mounted to the root /, using the pivot_root system call

  - Call the /sbin/init executable of the final root file system

  - Remove the initrd image from memory

initrd
How it works: Old change_root mechanism

- If the root device is not /dev/ram0 : The /dev/ram0 file system is mounted at the root /
- The file /linuxrc is executed if it exists
- When it finishes (or if it does not exist), the kernel mounts the final root file system instead of /dev/ram0 and remount it in the initrd directory (if it exists)
- The file /sbin/init is executed

initrd
Conclusion

- The initrd mechanism is not used anymore in favour of initramfs
- More information:

  - initrd(4) man page

  - Kernel source file init/do_mounts_initrd.c

  - Documentation file

  Documentation/admin-guide/initrd.rst

initramfs Functoning

- When the kernel is compiled, a compressed initramfs image is generated and integrated with the kernel image

- This default image is empty but the CONFIG_INITRAMFS_SOURCE configuration option can indicate to the kernel build process either choice: an existing cpio image to use, a directory containing the files to be included in the image or a configuration file indicating how to generate this image

initramfs
Functoning (cont.)

- The kernel implements a file system rootfs, a system based on ramfs or tmpfs and cannot be unmounted
- It decompresses the archive linked with it in this file system
- If another initramfs image has been passed through the bootloader, the kernel also decompresses it into the rootfs

  (thus possibly overwriting the files contained in the kernel linked image)
- The kernel calls the file /init (with the identifier user 0). This executable is not supposed to return

initramfs
Functoning (cont.)

- This /init executable traditionally performs the following actions:

  - Prepares to mount the root file system

  - Deletes the contents of rootfs

  - Mounts the final root file system

  - Then replace the root with this file system (with mount -move)

  - Starts the execution of /sbin/init on the new

initramfs
Functoning (cont.)

- The initramfs image can also contain the entire file system, especially in the case of small embedded systems
- In this case, the /init file can simply be used to start the necessary services and applications and not mount other file systems
- We will see in the lab how to generate such an image by hand using BusyBox

initramfs
Conclusion

- Advantages compared to initrd

  - Image integrated in the kernel (still separate for initrd)

  - Generally smaller image (archive vs. File system image)

  - very concise decompression code in the kernel and does not require a filesystem driver (except rootfs)

- More information: Documentation/filesystems/ramfs-rootfs-initramfs.rst

Plan

- Bootloader
- The interface between the bootloader and the kernel
- Decompressing of the kernel
- Booting the kernel
- Loading an initial memory file system
- Init, the first process
- References

init
Presentation

- init is the first process launched by the kernel
- It runs with the process ID (PID) 1
- It is launched (see function kernel_init in init/main.c) from the file :
  - Indicated by the init argument passed to the kernel
  - /sbin/init
  - /etc/init
  - /bin/init
  - /bin/sh
- The kernel panics if it is not able to find it (Kernel panic - not syncing: No working init found.)

init
Presentation

- Its standard input (file descriptor 0), its standard output (1) standard output (1) and its error output (2) are /dev/console
- It is a daemon whose lifetime is that of the (it must not stop before, otherwise the Kernel panics: Kernel panic - not syncing:

  Attempted to kill init!)

init
Presentation

- It is the ancestor (direct or indirect) of all the processes running on the system (they are all launched by a fork from init or one of its descendants)
- All the zombie processes (terminated and waiting by their parent) whose parents terminate without having acknowledged at the end of their children (by a call to wait), are attached to init which acknowledges their termination
- It is responsible for initializing the system and, when the time comes, to stop the different services during shutdown

init
Presentation

- In Linux systems (embedded or not), there are mainly three variants (others exist but are rarer).

  - BusyBox

  - SysV-style init

  - systemd, used by most distributions for a few years

Busybox init

- BusyBox provides a simple implementation of init
- At startup, by default, it will execute the file /etc/init.d/rcS
- This file (most often a shell script) will all the initialization
  - Mounting of the file systems (most often by a mount -a using the /etc/fstab file)
  - Configuration of the network interfaces
  - Start of services...

SysV-style init
Levels of execution

- A runlevel indicates the state of the machine.
- The configuration of init tells it which services it must start/stop when passing from one run level to another
- There are 7 runlevels numbered from 0 to 6 (an eighth one, s or S is sometimes an alias for one of them)
- The telinit command allows the super user to switch from one execution level to another

SysV-style init
Levels of classic execution

- 0 : System shutdown (standard)
- 1 : Single-user mode (standard)
- 3 : Normal multi-user mode (Linux Standard Base)
- 5 : Normal multi-user mode + graphical interface (Linux Standard Base)
- 6 : System reboot (standard)

SysV-style init
etc/inttab

- init is based on the configuration file /etc/inittab
- In this file, the format of the lines is as follows: id:runlevels:action:process
  - id : unique identifier of the line
  - runlevels : list of runlevels for which the action must be done
  - action : action to be done (see list)
  - process : process to execute (if any)

SysV-style init
etc/inttab, actions

- initdefault : indicates the default execution level at startup (the process field is ignored)
- sysinit, boot and bootwait : process executed at system startup (the runlevels field is ignored)
- wait : starts the process and waits for it to ends
- respawn : starts the process (and restarts it as soon as it ends)
- ctrlaltdel : process executed when init receives the SIGINT signal, typically when the user ctrl+alt+delete in a console

SysV-style init
etc/inttab, example

```
# Start runlevel
id :2: initdefault :
# System initialization
si :: sysinit :/ etc / init .d/ rcS
# Single user mode
~:S: wait :/ sbin / sulogin
# /etc / init .d executes S et K scripts when the
# runlevel changes
l0 :0: wait :/ etc / init .d/rc 0
l1 :1: wait :/ etc / init .d/rc 1
l2 :2: wait :/ etc / init .d/rc 2
l3 :3: wait :/ etc / init .d/rc 3
l4 :4: wait :/ etc / init .d/rc 4
l5 :5: wait :/ etc / init .d/rc 5
l6 :6: wait :/ etc / init .d/rc 6
```

SysV-style init
Conclusion

- The initialization system from System-V is very simple to understand (even if the syntax of inittab is quite strange at first)
- Its big flaw is the lack of parallelization during services (services are started one after the other) and the lack of dependencies (service x needs service y).
- These dependencies have to be managed manually, by adjusting the link numbers in /etc/rc.d/rcx.d

SysV-style init
Conclusion

- Several alternatives have been developed and systemd has progressively imposed itself in the majority of general-purpose Linux distributions in the last few years
- Because of its simplicity, the SysV-style init is still used for example in embedded systems

Plan

- Bootloader
- The interface between the bootloader and the kernel
- Unpacking of the kernel
- Booting the kernel
- Loading an initial memory file system
- Init, the first process
- References

References

[1] Intel. **HPS SoC Boot Guide - Cyclone V SoC Development Kit.**
https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/an/an709.pdf, January 2016.